

MODULE 08/09

Sécurité & OAuth 2.0

Protéger son app et ses APIs

■ Durée : 1 semaine

■ Projet : Social Seller

■ Niveau : Débutant →
Opérationnel

Objectifs de la semaine

- ✓ Comprendre les JWT : structure, signature, validation
 - ✓ Implémenter le flow OAuth 2.0 de A à Z (comme WhatsApp/Facebook dans Social Seller)
 - ✓ Valider les signatures de webhooks pour éviter les faux événements
 - ✓ Configurer CORS, rate limiting et les headers de sécurité
 - ✓ Stocker les tokens et secrets correctement (jamais en clair, jamais en logs)
-

Planning de la semaine

JOUR 1	JWT et authentification	2-3h
• JWT : header.payload.signature — lire et décoder un token • Algorithmes : HS256 (secret partagé) vs RS256 (clé publique/privée) • Claims : iss, sub, exp, iat — et claims custom (tenant_id, role) • Vérifier la signature et l'expiration côté serveur		
JOUR 2	OAuth 2.0 Authorization Code Flow	2-3h
• Les 4 étapes : Authorization Request, User Consent, Code Exchange, Token • State parameter : protection anti-CSRF • PKCE : protection supplémentaire pour apps mobiles • Access token vs Refresh token — durées de vie		
JOUR 3	Webhooks et signatures	2-3h
• Qu'est-ce qu'un webhook ? Push vs Poll • Vérification HMAC-SHA256 : recréer la signature et comparer • timingSafeEqual : éviter les timing attacks • Idempotence : déduplication des événements (webhook_events table)		
JOUR 4	CORS, rate limiting, headers	2-3h
• CORS : pourquoi et comment le configurer strictement • Rate limiting : protéger contre le DDoS et le brute force • Headers de sécurité : X-Content-Type-Options, X-Frame-Options... • Variables d'environnement : ne jamais hardcoder un secret		
JOUR 5	Mini-projet de la semaine	3-4h
• Implémenter un mini-flow OAuth 2.0 complet avec GitHub : • Bouton 'Connexion avec GitHub', génération du state anti-CSRF		

- Callback : vérifier le state, échanger le code contre un token
- Sauvegarder le token chiffré (AES-256-GCM comme Social Seller)
- Afficher le profil GitHub de l'utilisateur connecté

Concepts clés détaillés

JWT — anatomie d'un token

```
// Un JWT = 3 parties séparées par des points, encodées en Base64
// eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyLTEyMyIsInRlbmFudCI6InQtNDU2In0.abc
// HEADER PAYLOAD SIGNATURE

// Header (décodé) :
{ "alg": "HS256", "typ": "JWT" }

// Payload (décodé) :
{
  "sub": "user-123",
  "tenant_id": "t-456",
  "role": "merchant",
  "iat": 1704067200,
  "exp": 1704672000
}

// Vérification côté backend (middleware Social Seller)
import jwt from 'jsonwebtoken';

const payload = jwt.verify(token, process.env.SUPABASE_JWT_SECRET!) as JwtPayload;
// Si la signature est invalide ou le token expiré → exception
```

OAuth 2.0 — le flow complet

```
// ÉTAPE 1 : Générer un state anti-CSRF et stocker en base
const state = crypto.randomBytes(32).toString('hex');
await db.insert({ state, tenantId, platform: 'facebook', expiresAt });

// ÉTAPE 2 : Rediriger vers Meta
const url = new URL('https://graph.facebook.com/v21.0/dialog/oauth');
url.searchParams.set('client_id', META_APP_ID);
url.searchParams.set('redirect_uri', CALLBACK_URL);
url.searchParams.set('state', state);
```

```

url.searchParams.set('scope', 'pages_messaging,pages_show_list');
// → L'utilisateur voit la page de consentement Meta

// ÉTAPE 3 : Callback – vérifier le state
const { code, state: receivedState } = req.query;
const stored = await db.consumeState(receivedState); // supprime de la DB
if (!stored) throw new Error('Invalid state – possible CSRF attack');

// ÉTAPE 4 : Échanger le code contre un token
const tokenRes = await fetch('/oauth/access_token', {
  params: { client_id, client_secret, code, redirect_uri }
});
const { access_token } = await tokenRes.json();

```

Validation de signature webhook

```

// Social Seller – backend/src/services/metaGraphClient.ts
import { createHmac, timingSafeEqual } from 'node:crypto';

export function verifyMetaSignature(
  rawBody: Buffer,
  signatureHeader: string | undefined
): void {
  // 1. Recalculer la signature attendue
  const expected = createHmac('sha256', META_APP_SECRET)
    .update(rawBody)
    .digest('hex');

  // 2. Extraire la signature reçue (format: 'sha256=abc123...')
  const provided = signatureHeader?.slice('sha256='.length) ?? '';

  // 3. Comparaison à temps constant (évite timing attacks)
  const expectedBuf = Buffer.from(expected, 'hex');
  const providedBuf = Buffer.from(provided, 'hex');

  if (expectedBuf.length !== providedBuf.length ||
    !timingSafeEqual(expectedBuf, providedBuf)) {
    throw new WebhookSignatureError();
  }
}

```

Chiffrement AES-256-GCM (tokens OAuth)

```
// Ne jamais stocker un token OAuth en clair en base
import { createCipheriv, createDecipheriv, randomBytes } from 'crypto';

const KEY = Buffer.from(process.env.TOKEN_ENCRYPTION_KEY!, 'base64');

export function encryptToken(token: string): string {
  const iv = randomBytes(12); // 96 bits pour GCM
  const cipher = createCipheriv('aes-256-gcm', KEY, iv);
  const encrypted = Buffer.concat([
    cipher.update(token, 'utf8'),
    cipher.final()
  ]);
  const tag = cipher.getAuthTag(); // Tag d'authenticité
  // Stocker iv + tag + données chiffrées ensemble
  return Buffer.concat([iv, tag, encrypted]).toString('base64');
}
```

■ Règle d'or : les secrets (API keys, tokens, passwords) ne vivent que dans les variables d'environnement. Jamais dans le code, jamais dans les logs, jamais dans Git. Générer `TOKEN_ENCRYPTION_KEY` avec : `openssl rand -base64 32`

Exercices pratiques

Exercice 1 — Décoder un JWT (20 min)

- Aller sur jwt.io
- Coller le JWT de ta session Supabase (récupéré via `supabase.auth.getSession()`)
- Identifier : sub (user id), role, exp, tenant_id dans les claims custom
- Vérifier la signature avec ta `SUPABASE_JWT_SECRET`

Exercice 2 — Middleware auth complet (45 min)

- Créer un middleware Express qui extrait et vérifie le JWT Supabase
- Ajouter `req.user = { id, tenantId, role }` après vérification
- Tester : requête sans token → 401, avec token expiré → 401, avec token valide → 200

Exercice 3 — Vérifier un webhook (45 min)

- Créer une route POST `/webhook/test`
- Implémenter la vérification HMAC-SHA256 avec un secret statique
- Tester avec curl en envoyant une signature correcte puis incorrecte

Ressources pour aller plus loin

- JWT.io : jwt.io — déboguer et visualiser les JWT
- OWASP Top 10 : owasp.org/www-project-top-ten
- OAuth 2.0 Simplified (livre) : oauth.net/2/
- Meta Webhook docs : developers.facebook.com/docs/messenger-platform/webhooks
- Supabase Auth docs : supabase.com/docs/guides/auth